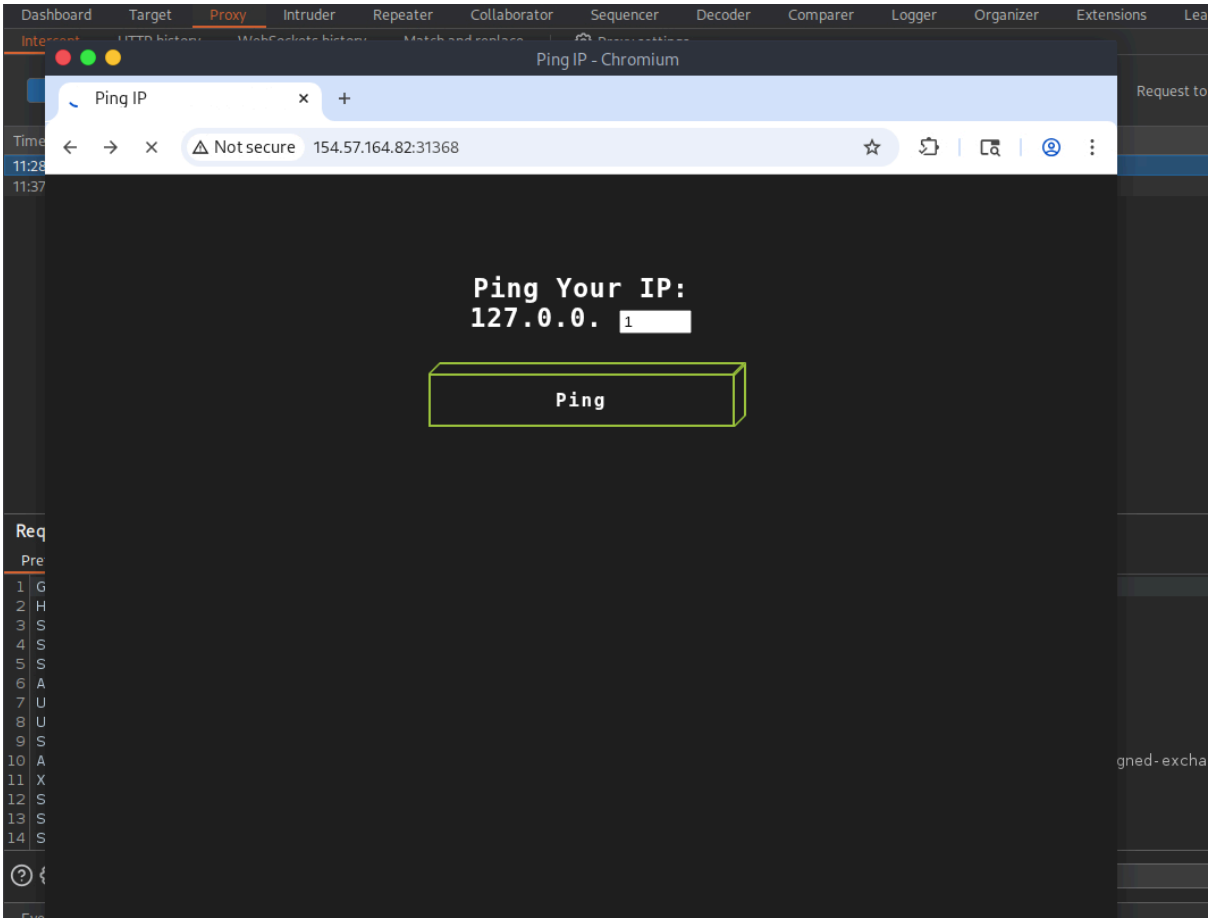
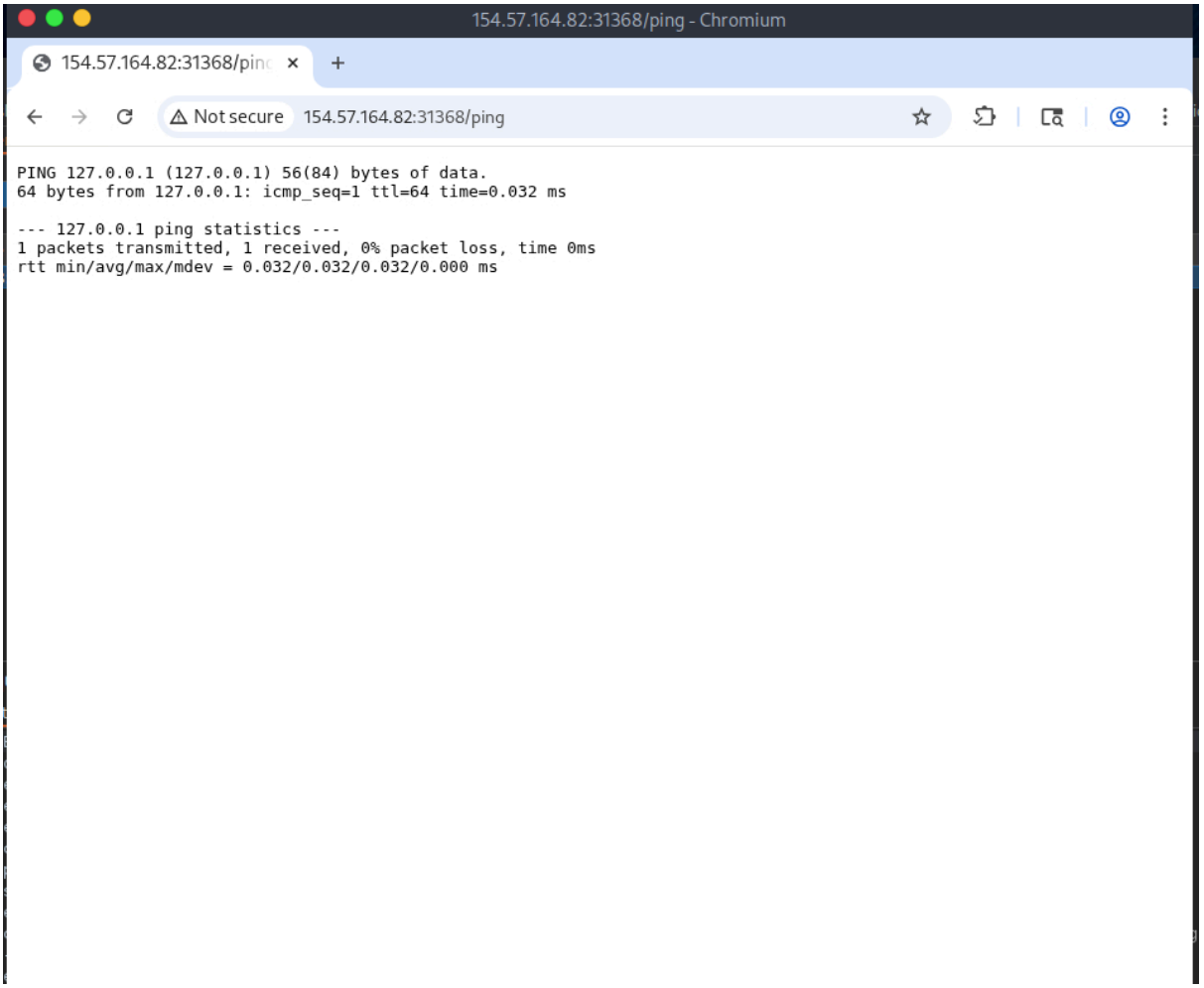
Try intercepting the ping request on the server shown above, and change the post data similarly to what we did in this section. Change the command to read 'flag.txt'

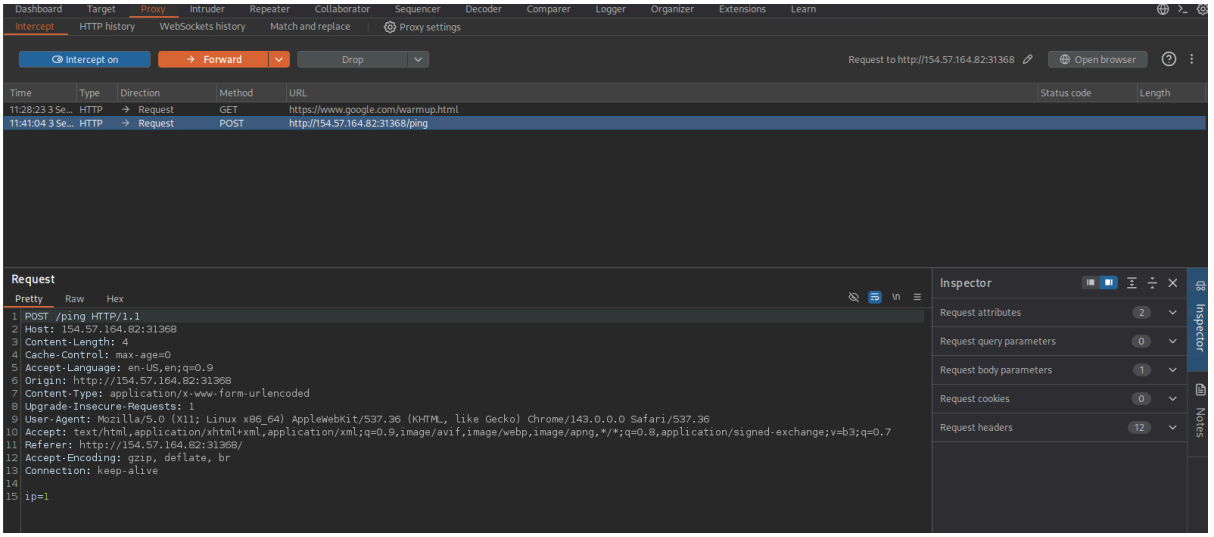
Więc uruchomiłem target i następnie uruchomiłem Burp Suite. Przeklikałem by mieć dostęp do sekcji Proxy i uruchomiłem przeglądarkę. WPisałem ip i przepuściłem request do serwera.



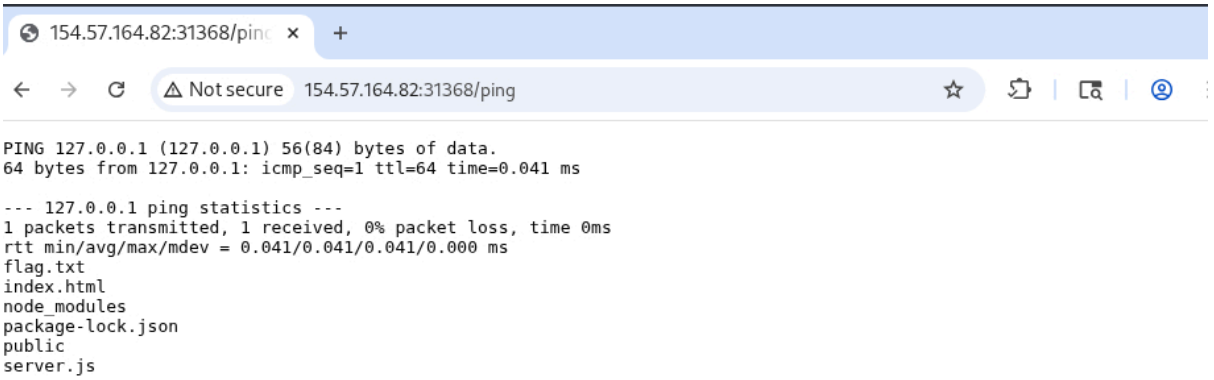
Przy okazji wysłałem zapytanie o ip 127.0.0.1 i przepuściłem.



No co się stanie jak wyślemy zapytanie z komendą która nam powie jakie pliki są dostępne do odczytu?
No niestety nie możemy wpisać na stronie literek ale możemy manipulować wysłanymi zapytaniami.

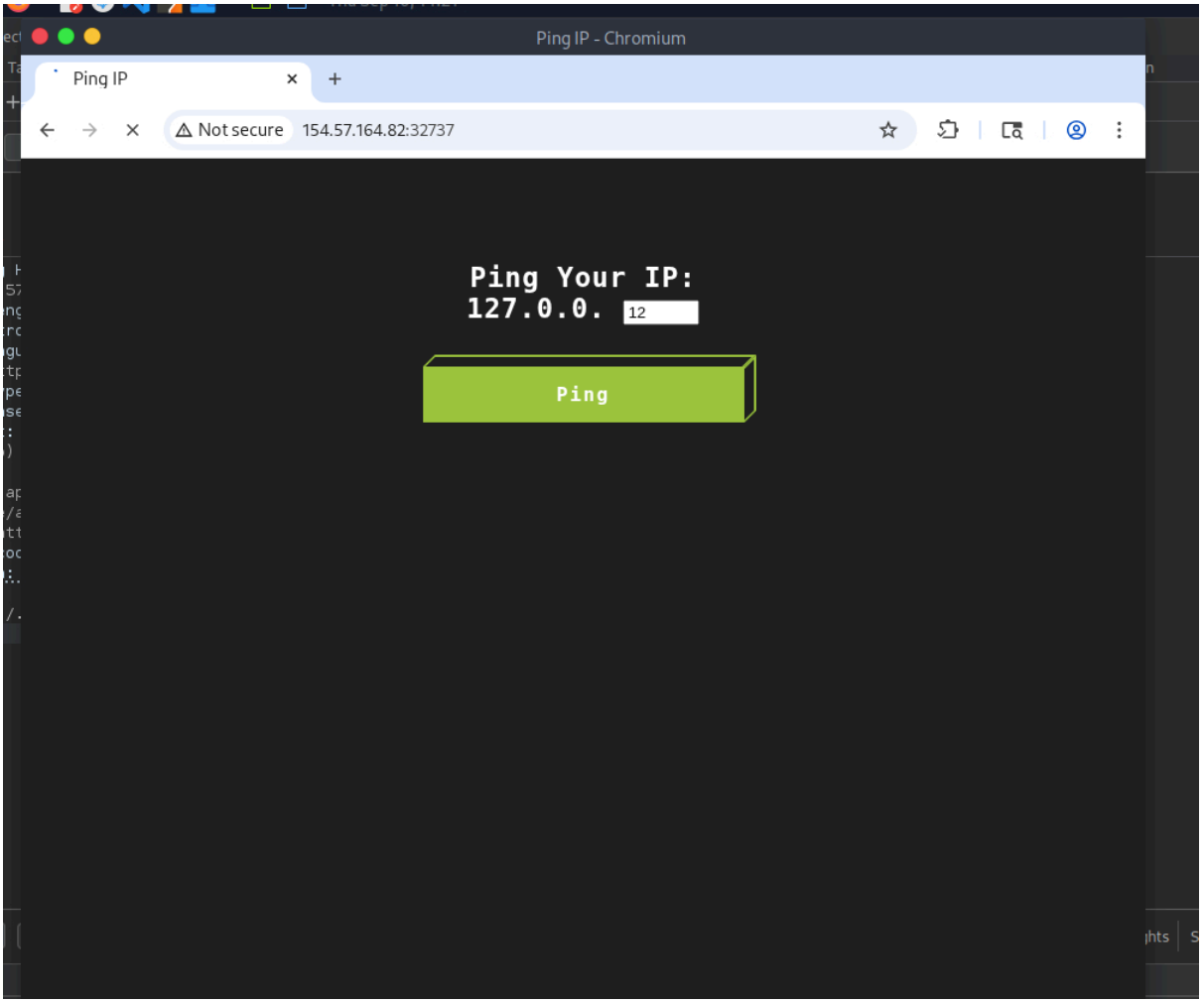
1. Wysyłamy zwykłe zapytanie z 1
2. Szukamy zmiennej, u nas to jest 1 i następnie wpisujemy ; ls;
3. 

4. Przepuszczamy

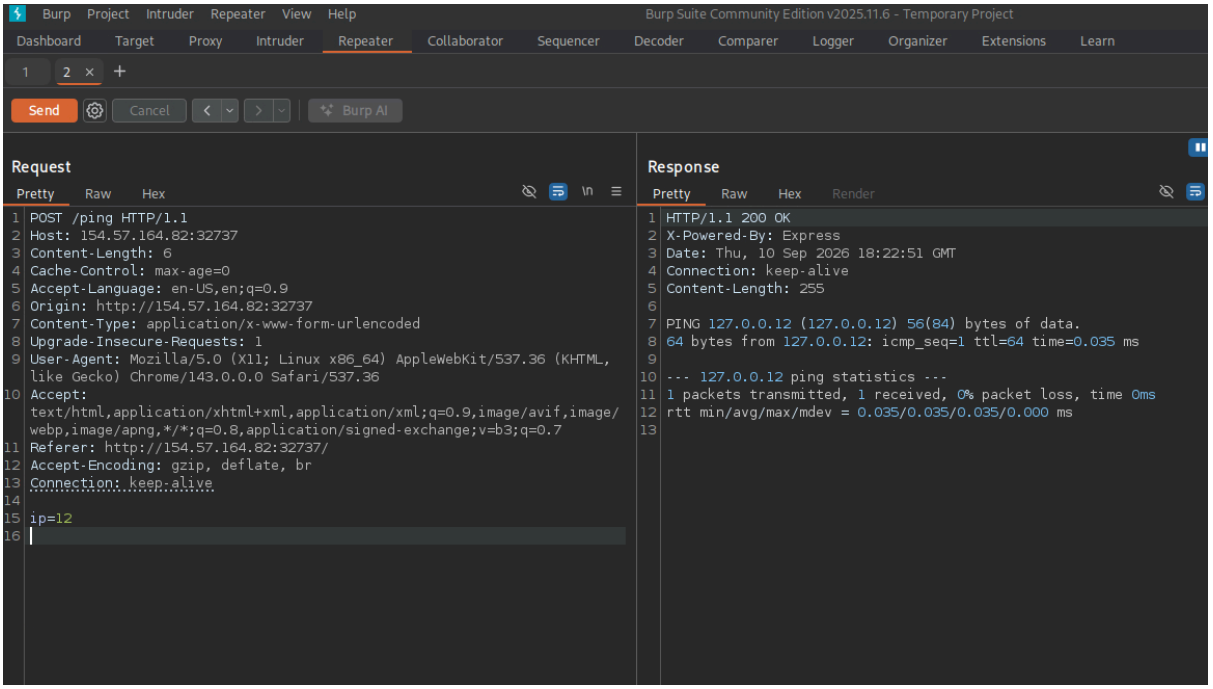


5.
6. No i widzimy listę
7. W takim razie możemy odczytać naszą flagę wpisując “cat i nazwa pliku z rozszerzeniem”

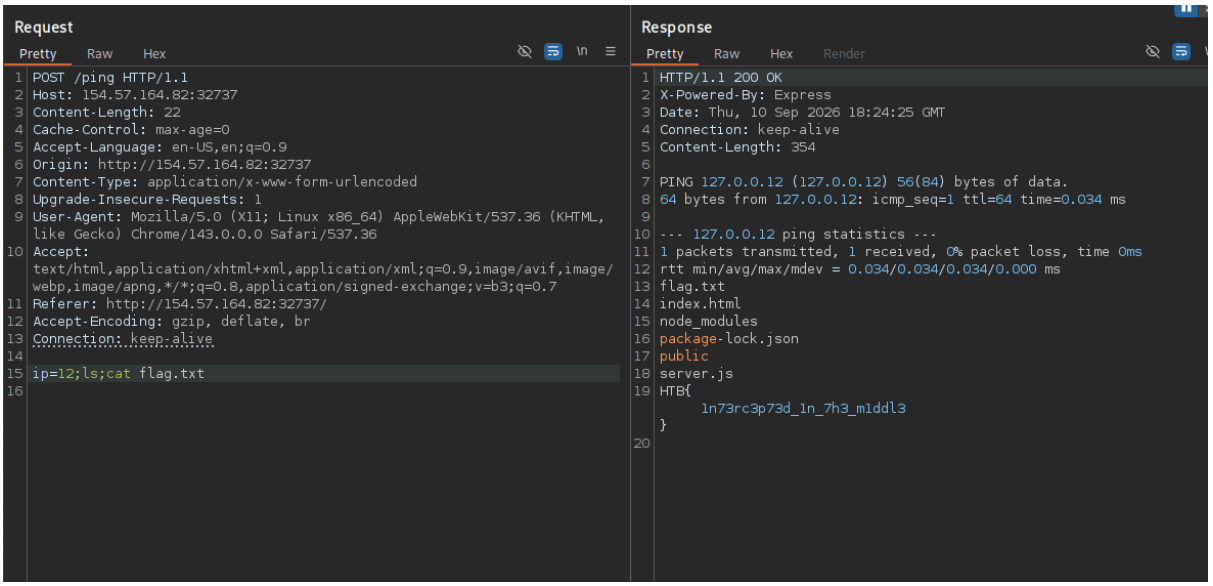
DObra mamy za zadanie znaleźć flagę za pomocą repeatera w Burp Suite.
uruchomiłem stronę



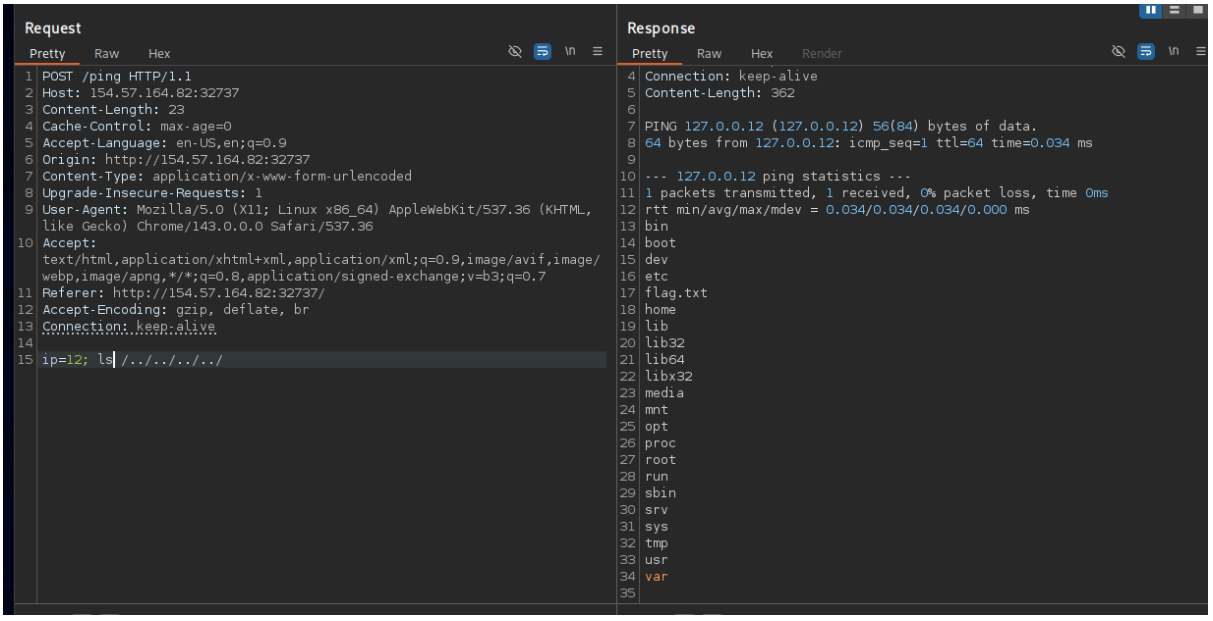
następnie historię zapytań przesłałem do repeatera za pomoca ctr + R



po wpisaniu komendy zamiast numeru ip możemy odczytać co jest w na tym serwerze jakie pliki.
co ciekawe będzie flaga ale…



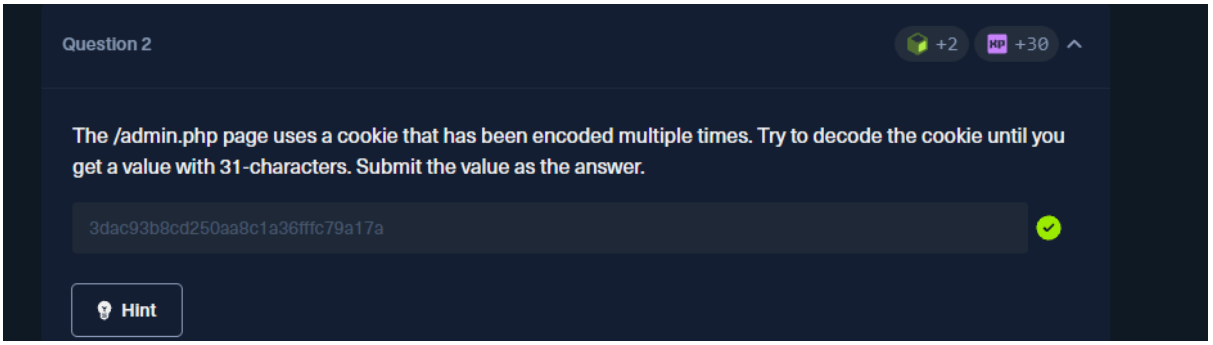
flaga jest ale nie ta
jak klikniemy w zarowke/podpowiedź to dowiemy się że to nie ten folder
Głowiłem się i znalazłem odpowiedź na forum.
Trzeba było wyjść do najwyższych pod folderów i tam była nasza odpowiedź



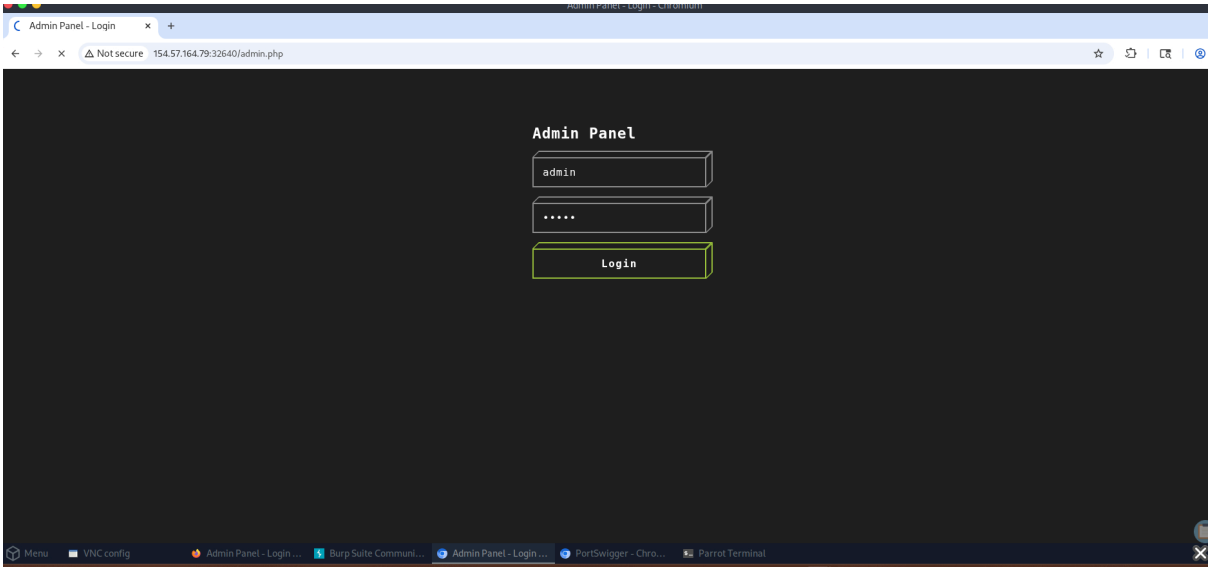
po wklepaniu cat plus plik zobaczymy realna odpowiedź

The string found in the attached file has been encoded several times with various encoders. Try to use the decoding tools we discussed to decode it and get the flag.

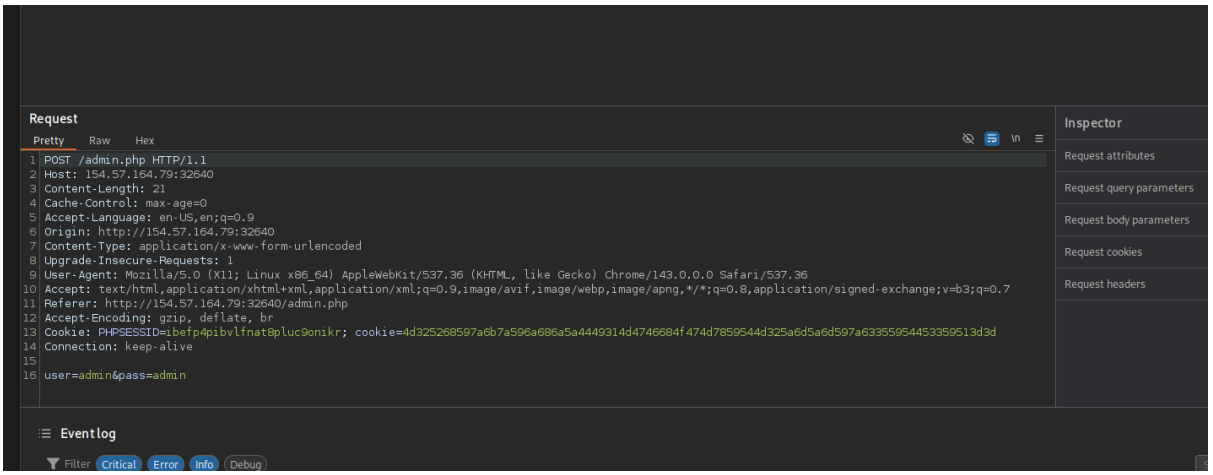
przy tym zadaniu trzeba było użyć miejsca tam gdzie jest iop i wklepać do zmiennej zawartość flagi i potem przeklikać po prawej stronie (w inspector -> request body parameter) i samo będzie odkodowane i zobaczysz flagę



tak wygląda strona:



po wklepaniu czegoś zobaczyłem zawartość zapytania w Burp Suite Proxy -> interecept skopiowałem zawartość cookie i wkleiłem do gemini by sprawdzić co to za enkodowanie i mi rozwiązało zadanie



Pokaż kod

To, co podałeś, w ścisłym znaczeniu nie jest "szyfrowaniem" (które wymaga klucza do odszyfrowania), lecz ciągiem **podwójnego kodowania**.

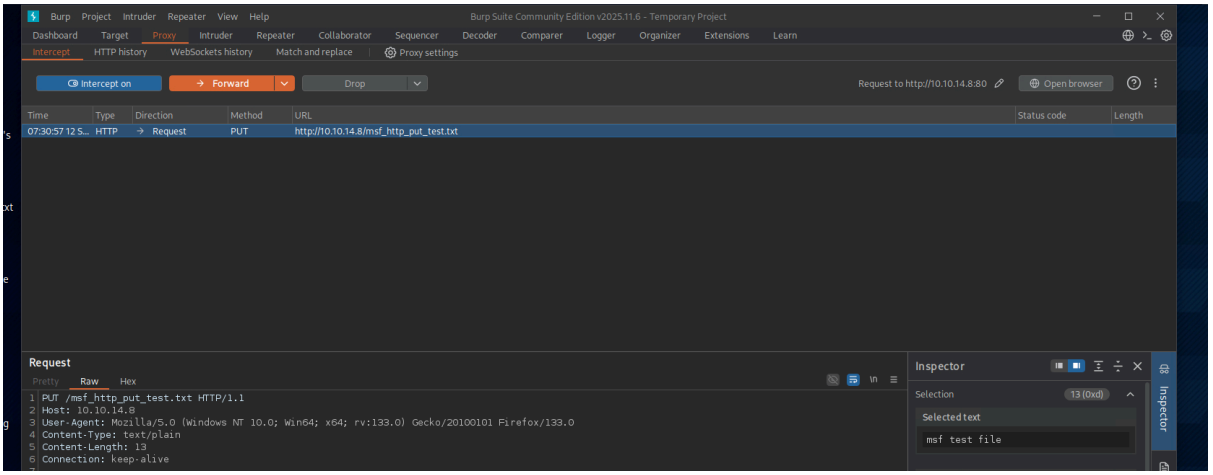
Kiedy zdekoduje się ten ciąg krok po kroku, mechanizm wygląda następująco:

Krok 1: Kodowanie szesnastkowe (Hexadecimal)
Twój ciąg składa się wyłącznie ze znaków z zakresu 0-9 i a-f. Jeśli zdekodujemy go z formatu szesnastkowego (Hex) na zwykły tekst (ASCII), otrzymamy:
`M2RhYzkzYjhjZDI1MGFhOGMxYTM2ZmZmYzc5YTE3YQ==`

Krok 2: Kodowanie Base64
Ciąg uzyskany w poprzednim kroku, ze względu na charakterystyczne dwa znaki równe (`==`) na końcu i użyty alfabet, to klasyczne kodowanie Base64. Zdekodowanie go daje nam ostateczny wynik:
`3dac93b8cd250aa8c1a36fffc79a17a`

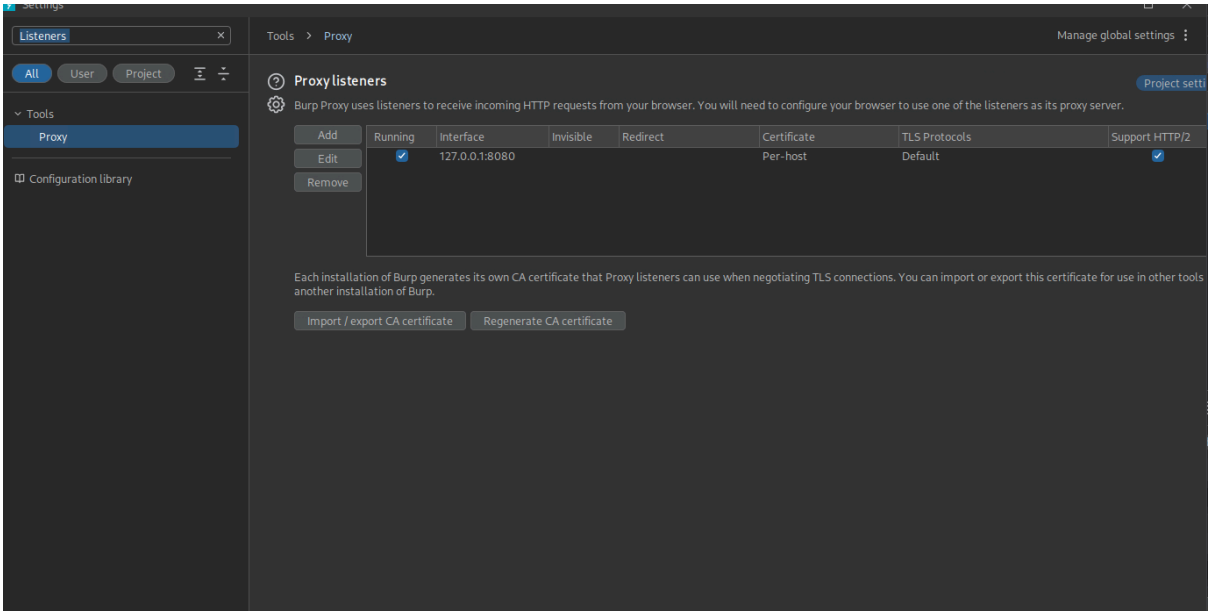
Co to jest ten wynikowy ciąg?
Uzyskany tekst (`3dac93b8cd250aa8c1a36fffc79a17a`) to 31-znakowy ciąg szesnastkowy. Ze względu na swój format i długość, z największym prawdopodobieństwem jest to **funkcja skrótu (hash)**, konkretnie **MD5**. Hashe MD5 mają zazwyczaj 32 znaki, więc bardzo możliwe, że brakuje w nim zera na samym początku (np. `03dac9...`), które mogło zostać obcięte przez jakiś system podczas zapisu. Hashe są jednokierunkowe – nie da się ich wprost "odszyfrować", służą zazwyczaj do weryfikacji integralności plików lub przechowywania haseł.

Podsumowując:
Twój ciąg to hash (prawdopodobnie **MD5**) najpierw zakodowany w **Base64**, a następnie to Base64 zakodowano za pomocą **systemu szesnastkowego (Hex)**.

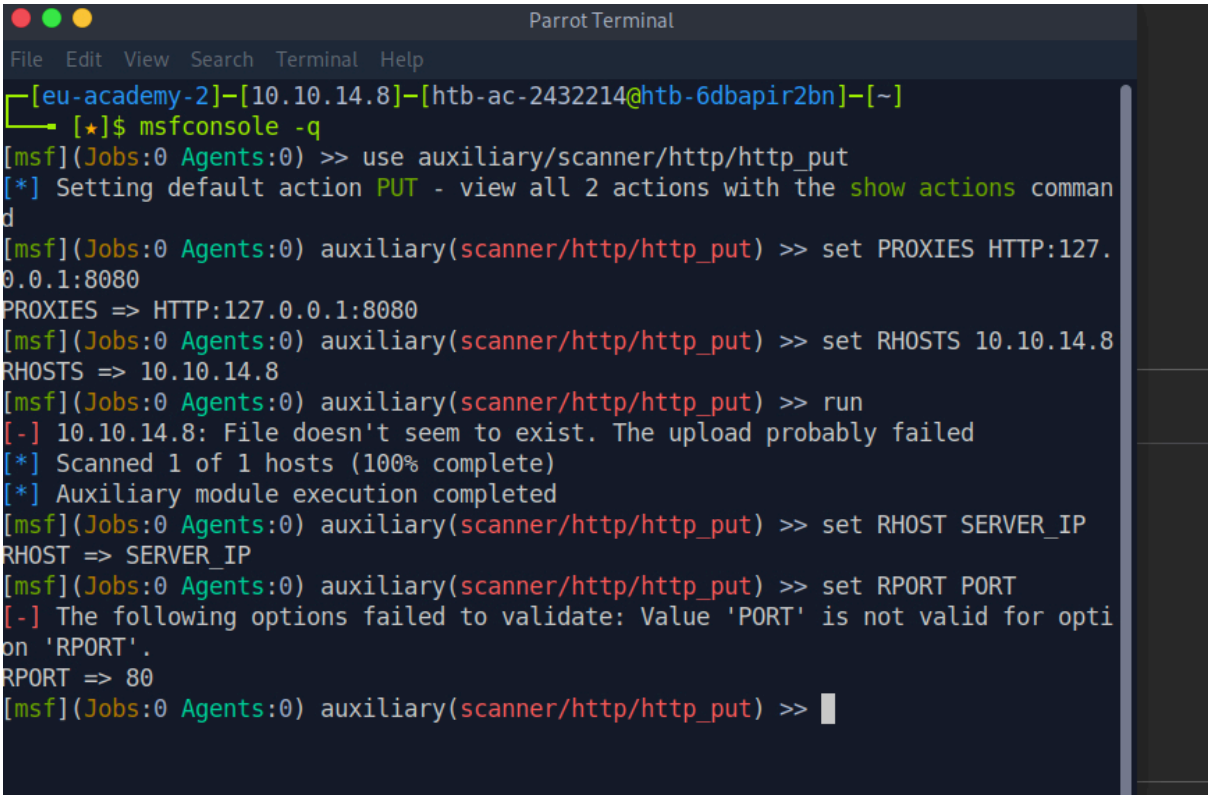


Try running 'auxiliary/scanner/http/http_put' in Metasploit on any website, while routing the traffic through Burp. Once you view the requests sent, what is the last line in the request?

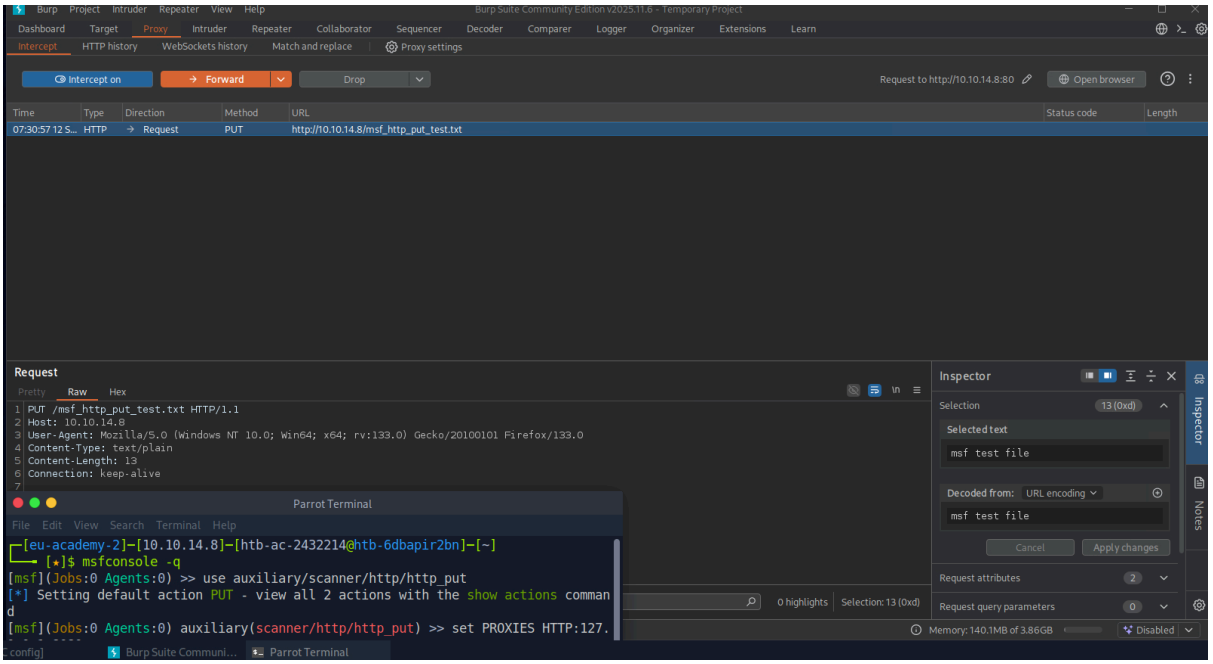
Jak to zrobiłem?
Sprawdziłem czy Burp Suite jest skonfigurowany



Następnie uruchomiłem metasploita i skonfigurowałem po czym go uruchomiłem.



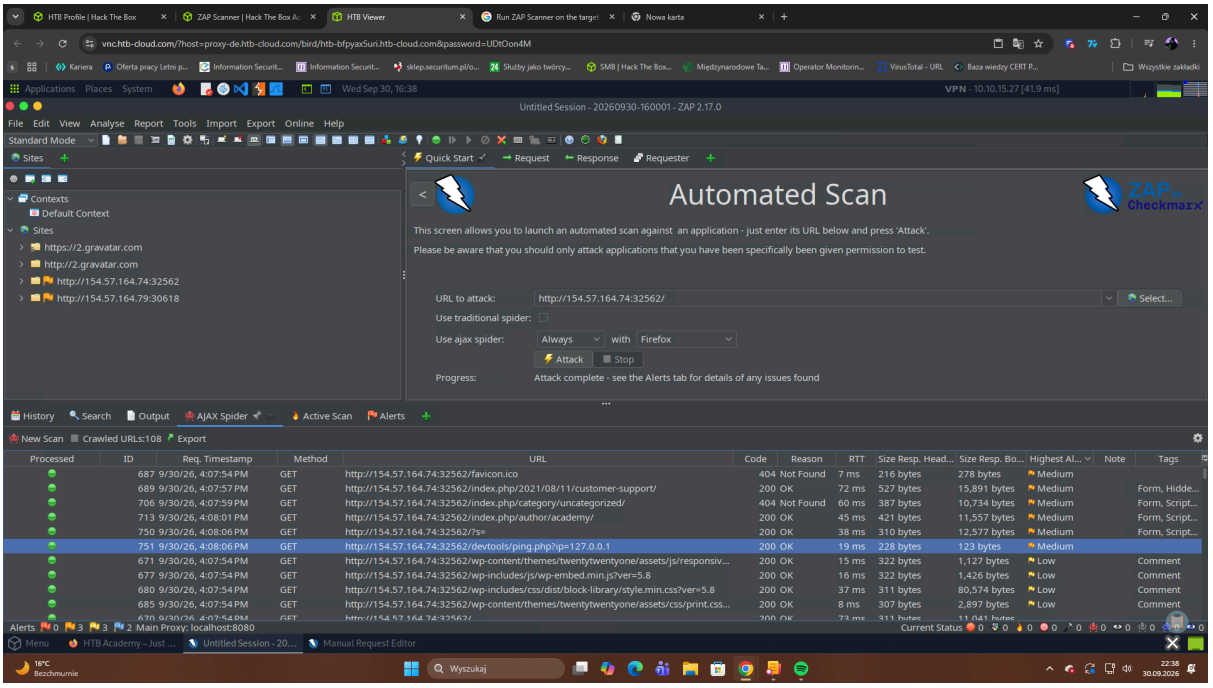
uruchomiłem i wynik był taki jaki prosili ;)



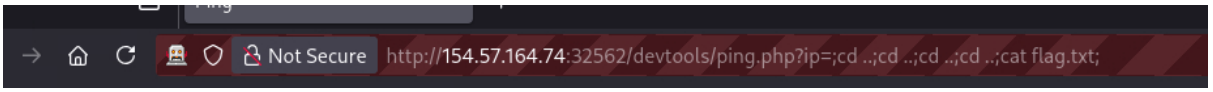
odpowiedz jet na dole zasłonięta

Run ZAP Scanner on the target above to identify directories and potential vulnerabilities. Once you find the high-level vulnerability, try to use it to read the flag at '/flag.txt'

Uruchomiłem zapa i opaliłam Ajax crawlera



Zobaczyłem pewien zapis i zauważyłem że ta podatność już była to wykorzystałem

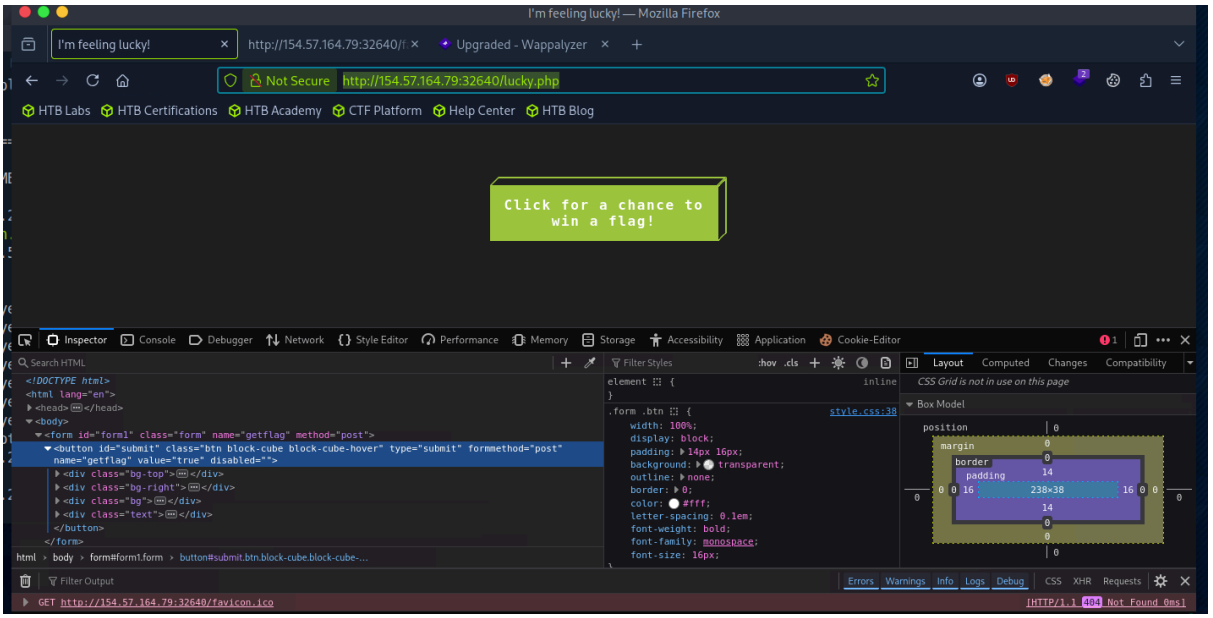


The /lucky.php page has a button that appears to be disabled. Try to enable the button, and then click it to get the flag.

jak do tego podszedłem?

Odpaliłem w przegladarce stronke : http://154.57.164.79:32640/lucky.php

Zobaczyłem że nie mogę kliknąć przycisku:



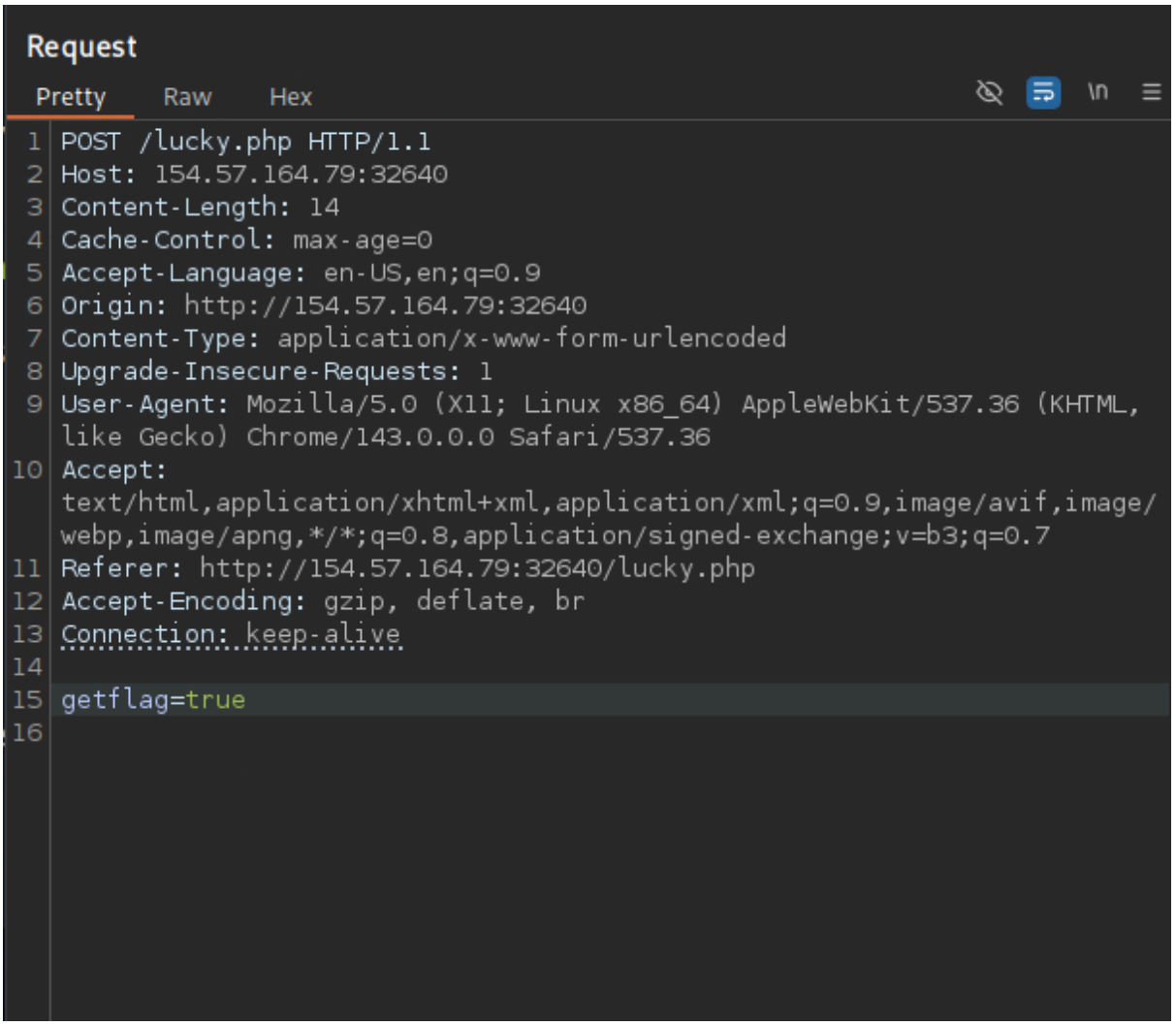
Zajrzałem w dev tools w firefoxie i zobaczyłem ze jest zapis disabled to go usunąłem:



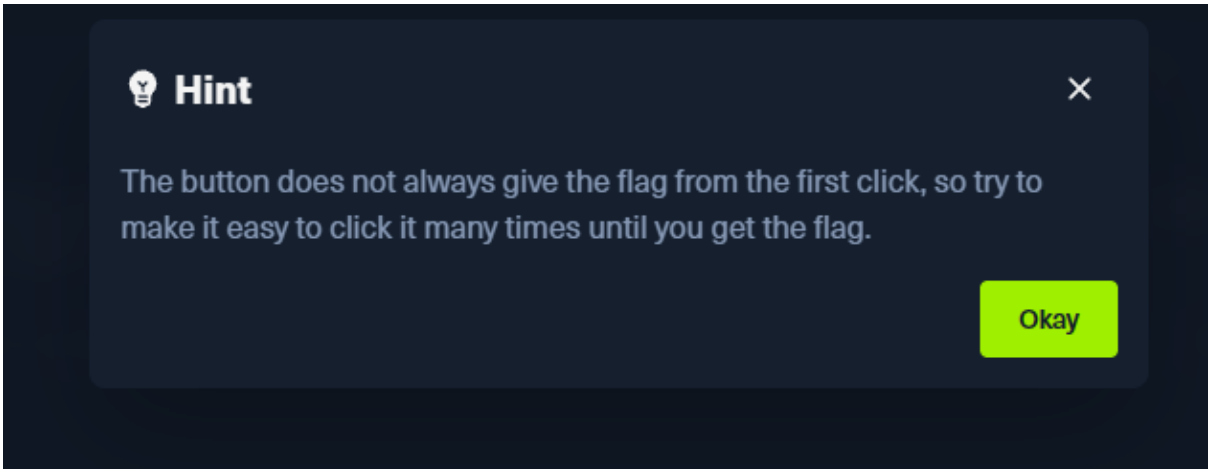
następnie zobaczyłem że nic się nie zmienia:

To postanowiłem zrobić to za pomocą Burp Suite:

Kliknąłem przycisk i zobaczyłem jak wygląda request po modyfikacji:



To jest ekran w sekcji Repeater. Następnie zacząłem się zastanawiać o co chodzi i znalazłem HINT



Więc próbowałem klikać ten request i nic
Zacząłem szukać po necie i ściągnęłam program który wyklikuje automatycznie i podaje

odpowiedź. Tak to mniej wiecej wygladalo bez odpowiedzi;

```
┌─[eu-academy-2]─[10.10.15.239]─[htb-ac-2432214@htb-tlootbvwgz]─[~]
└──╼ [★]$ python3 hitbutton.py http://154.57.164.79:32640/lucky.php
[*] Targeting : http://154.57.164.79:32640/lucky.php
[*] Max attempts: 8

[-] Attempt 1/8 — no flag yet (HTTP 200)
[-] Attempt 2/8 — no flag yet (HTTP 200)
[-] Attempt 3/8 — no flag yet (HTTP 200)
[-] Attempt 4/8 — no flag yet (HTTP 200)
[-] Attempt 5/8 — no flag yet (HTTP 200)
[-] Attempt 6/8 — no flag yet (HTTP 200)
[-] Attempt 7/8 — no flag yet (HTTP 200)
```

Once you decode the cookie, you will notice that it is only 31 characters long, which appears to be an md5 hash missing its last character. So, try to fuzz the last character of the decoded md5 cookie with all alpha-numeric characters, while encoding each request with the encoding methods you identified above. (You may use the "alphanum-case.txt" wordlist from Seclist for the payload)

teraz znalazłem sciezke do pliki txt

Plik `alphanum-case.txt` znajdziesz w popularnym pakiecie narzędzi do testów bezpieczeństwa (fuzzingu) o nazwie SecLists.

Oto główne miejsca, w których możesz go pobrać lub znaleźć bezpośrednio na swoim systemie:

🌐 Gdzie znaleźć online?

- **GitHub (SecLists):** Oficjalny plik znajduje się w repozytorium SecLists na GitHubie. Jest on zlokalizowany w ścieżce `SecLists/Fuzzing/alphanum-case.txt`.
- **Alternatywne repozytorium (fuzzdb):** Bardzo podobną listę pod nazwą `wordlist-alphanumeric-case.txt` znajdziesz również w projekcie fuzzdb na GitHubie.

💻 Gdzie znaleźć lokalnie (np. w Kali Linux)?

Jeśli korzystasz z systemu dystrybucji do testów penetracyjnych, takich jak **Kali Linux**, i masz zainstalowany pakiet `seclists`, plik ten znajduje się w domyślnym katalogu z listami słów:

```bash
/usr/share/seclists/Fuzzing/alphanum-case.txt
```

Używaj kodu z rozwagą.

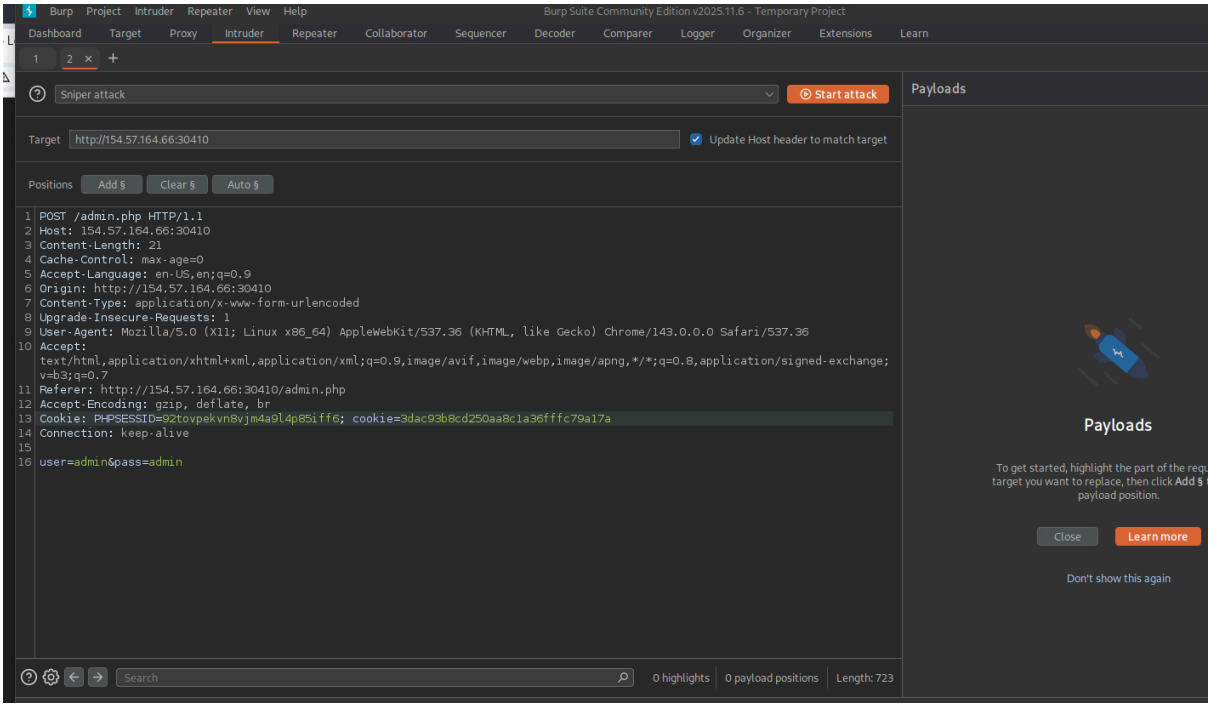
Jeśli jeszcze go nie masz, możesz zainstalować cały pakiet komendą:

GitHub
SecLists/F
master · da
File: SecList
danielmiessl

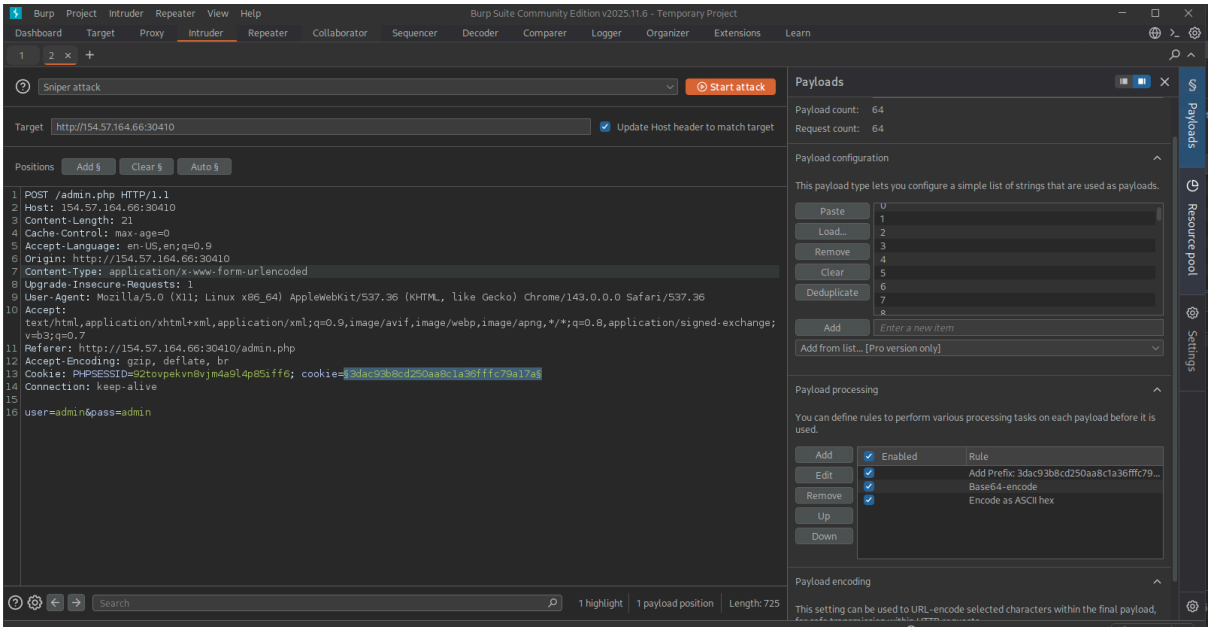
GitLab
Fuzzing/alp
d20b102ca
File: alphanu
seclists unde

GitHub
fuzzdb/wor
alphanume
File: fuzzdb-
misc/wordlis

jak teraz mamy plik to mozemy sie brac za fuzzowanie .itp



zastępuje cookie ciągiem 31 znaków z poprzedniego zadania



tak skonfigurowałem
po czym przeczesałem każdy request i znalazłem na dole strony rozwiązanie

You are using the 'auxiliary/scanner/http/coldfusion_locale_traversal' tool within Metasploit, but it is not working properly for you. You decide to capture the request sent by Metasploit so you can manually verify it and repeat it. Once you capture the request, what is the 'XXXXX' directory being called in '/XXXXX/administrator/..'?

Odpaliłem i włączyłem burp suite ale mi nie działało.
więc odpaliłem nasłuch w terminalu i mi działało a wcześniej skonfigurowałem metasploita tak jak prosili a oto wynik.

